TASKFLOW – INTERACTIVE TASK MANAGER

Project Documentation – Week 2

HTML5 • CSS3 • JavaScript • localStorage

1. Project Overview

TaskFlow is a responsive browser-based task management application developed using HTML5, CSS3 and JavaScript. The application allows users to create, update, complete, search, filter, sort and delete tasks while maintaining task data using browser localStorage. It also provides dark/light mode, task statistics, progress tracking, priority, category, due-date information and toast notifications.

2. Project Objectives

- Build a practical and user-friendly task management application.
- Apply JavaScript DOM manipulation and event handling.
- Implement persistent data storage using localStorage.
- Create a responsive interface for desktop, tablet and mobile devices.
- Implement search, filtering and sorting functionality.
- Provide form validation and user feedback.
- Practice modular frontend development using separate JavaScript files.

3. Key Features

Feature	Description
Add Task	Create a new task with priority, category and optional due date.
Edit Task	Update the text of an existing task.
Delete Task	Remove an individual task after confirmation.
Complete Task	Mark tasks as completed or active.
Search	Search tasks by their text.
Filter	View All, Active or Completed tasks.
Sort	Sort by newest, oldest, priority, due date or alphabetical order.
Statistics	Display total, active and completed task counts.
Progress	Show completion percentage using a progress bar.
Dark/Light Mode	Switch between light and dark themes and remember the preference.
localStorage	Persist tasks between browser sessions.
Toast Notifications	Display success, warning and error feedback.
Responsive UI	Adapt the layout for desktop, tablet and mobile screens.
Keyboard Shortcut	Ctrl + K focuses the search box and Escape clears search.

4. Technology Stack

Technology	Purpose
HTML5	Page structure and semantic UI elements
CSS3	Layout, responsive design, animations and themes
JavaScript	Application logic, DOM manipulation and events
localStorage	Client-side persistent task and theme storage
Git/GitHub	Version control and project hosting
Live Server	Local development and browser testing

5. Project Structure

```
week2-task-manager/
├── index.html
├── css/
│   ├── style.css
│   └── theme.css
├── js/
│   ├── app.js
│   ├── storage.js
│   ├── ui.js
│   └── utils.js
├── screenshots/
│   ├── desktop.png
│   ├── dark-mode.png
│   ├── search-filter.png
│   └── mobile.png
└── README.md
```

6. Application Architecture

The application follows a simple modular frontend architecture. app.js manages the main application state and user interactions. storage.js handles saving and loading data. ui.js is responsible for rendering tasks, statistics, progress, filters, badges and toast notifications. utils.js contains reusable helper functions such as task ID generation, validation and HTML escaping.

7. Application Workflow

1. The page loads and retrieves saved tasks from localStorage.

2. The saved theme is loaded and applied.

3. The UI renders the current task list and statistics.

4. The user can add, edit, delete or complete tasks.

5. Every important state change updates localStorage.

6. Search, filter and sorting modify the displayed task collection.

7. Statistics and progress are recalculated after UI refresh.

8. Toast messages provide feedback after user actions.

8. localStorage Implementation

Tasks are stored using the key "week2_tasks". JavaScript converts the task array into JSON before saving it and parses the JSON again when loading the application. The theme preference is stored separately using the key "week2_theme".

Example data model:

```
{
  id: 1750000000000,
  text: "Complete JavaScript assignment",
  completed: false,
  priority: "high",
  category: "study",
  dueDate: "2026-08-25",
  createdAt: "2026-08-22T10:00:00.000Z"
}
```

9. Task Rendering and UI Management

The renderTasks() function creates task cards dynamically. Before rendering, tasks are filtered according to the selected filter and search text, then sorted according to the selected sorting method. Each task card contains a completion checkbox, task text, metadata badges and Edit/Delete actions.

10. Search, Filter and Sort

Search performs a case-insensitive text comparison. Filtering separates tasks into All, Active and Completed groups. Sorting supports newest, oldest, priority, due date and alphabetical order. These operations are applied before the final task list is rendered.

11. Dark/Light Theme

The theme system toggles the dark class on the body element. CSS custom properties are used to change background, card, text, muted text and border colors. The selected theme is stored in localStorage so that it remains active after refreshing the page.

12. Validation and Error Handling

- Empty task text is rejected.
- Edited tasks cannot be empty.
- Delete and clear-completed operations require confirmation.
- Missing DOM elements are handled safely in UI helper functions.
- User-generated task text is escaped before being inserted into HTML.

13. Responsive Design

CSS media queries adjust the application for tablet and mobile screens. Input controls stack vertically on small devices, statistics become a single-column layout, task action buttons stack vertically, and search/sort controls adapt to narrower screens.

14. Accessibility

- ARIA labels are used for task checkboxes and action buttons.
- Keyboard focus styles are provided using :focus-visible.
- Search can be focused using Ctrl + K.
- Escape clears an active search.
- Reduced-motion preferences are respected using prefers-reduced-motion.

15. Testing Plan

Test Case	Action	Expected Result
Add task	Enter valid text and submit	Task appears in list and is saved
Empty task	Submit empty input	Validation message appears
Complete task	Click checkbox	Task becomes completed and progress updates
Edit task	Click Edit and provide new text	Task text updates
Delete task	Click Delete and confirm	Task is removed
Search	Enter a search term	Matching tasks are displayed
Filter	Select All/Active/Completed	Correct tasks are displayed
Sort	Change sort option	Tasks reorder correctly
Dark mode	Click theme button	Dark theme is applied and saved
Persistence	Refresh browser	Tasks remain available

16. A/B Testing Plan

A/B testing can be used to compare two UI variants and identify which design produces better usability. The recommended experiment is to compare the original task interface with the enhanced TaskFlow interface.

A/B Testing Element	Description
Variant A	Basic task input, simple filters and standard task cards
Variant B	Enhanced interface with priority/category/due date, search, sorting, progress, badges and toast feedback
Primary Metric	Task completion rate
Secondary Metrics	Time to add a task, number of completed tasks, search usage and user interaction rate
Goal	Determine whether the enhanced interface improves task organization and usability

17. Performance Considerations

The project is a lightweight client-side application with no external backend dependency. Tasks are stored locally, DOM updates are performed only when the application state changes, and reusable functions separate storage, UI and utility responsibilities.

18. Future Enhancements

- User authentication and cloud synchronization.
- Drag-and-drop task ordering.
- Task reminders and browser notifications.
- Recurring tasks.
- Calendar integration.
- Analytics dashboard.
- Backend API and database integration.
- Multi-device synchronization.
- Export/import tasks as JSON or CSV.

19. Conclusion

TaskFlow demonstrates the practical use of HTML5, CSS3 and JavaScript to build a complete interactive web application. The project combines CRUD-style task operations, client-side persistence, responsive UI design, search, filtering, sorting, progress tracking, theme management and user feedback. It provides a strong foundation for further development into a full-stack task management platform.

20. Screenshots

Add the following screenshots to the screenshots/ folder and insert them in this section when preparing the final submission:

- Dashboard / Main TaskFlow interface
- Adding a task with priority, category and due date
- Dark mode
- Search and filter functionality
- Responsive mobile layout